

CSE 433 Final Project

Jack Hunter

May 10, 2026

1 Model and Performance Analysis

My final single-model submission is `UltraWideScaledTail10`, implemented in `train_ultrawidescaledtail10_repro.py`. I designed the model for the project constraint of at most 10 weighted layers, where weighted layers are convolutional or linear modules on the forward path. Batch normalization, activation functions, pooling, channel gating, residual additions, stochastic depth, and scalar residual parameters are not counted as weighted layers.

1.1 Architecture

The model uses a wide convolutional trunk followed by a low-resolution scaled residual tail. The central idea is to spend the limited layer budget on high-capacity feature maps, while placing the extra residual refinement after the spatial resolution has been reduced. This gives the model high channel capacity without violating the 10-layer depth limit.

Count	Module	Type	Purpose
1	<code>stem</code>	Conv2d	Initial RGB projection
2	<code>block1.conv1</code>	Conv2d	Wide feature extraction
3	<code>block1.conv2</code>	Conv2d	Wide feature extraction
4	<code>block2.conv1</code>	Conv2d	768-channel feature block
5	<code>block2.conv2</code>	Conv2d	768-channel feature block
6	<code>block3.conv</code>	Conv2d	Low-resolution transition
7	<code>tail.conv1</code>	Conv2d	Grouped residual tail
8	<code>tail.conv2</code>	Conv2d	Grouped residual tail
9	<code>tail.conv3</code>	Conv2d	Dense residual tail
10	<code>fc</code>	Linear	CIFAR-10 classifier

The audited model has exactly 10 weighted layers and 21,337,656 parameters. A dummy forward pass returns logits of shape $(2, 10)$, matching the 10 CIFAR-10 classes.

1.2 Visual model data flow

Figure 1 shows the main data flow through the network for one CIFAR-10 image. The diagram reports the feature-map shape after each major stage, after the pooling operation inside that stage has been applied. The largest activation tensor appears early, before the spatial resolution has been reduced. The later tail operates on a compact $768 \times 3 \times 3$ tensor, which is why it is an efficient place to spend the remaining layer budget.

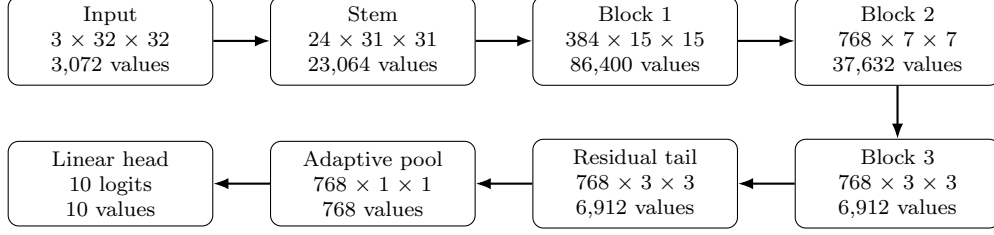


Figure 1: Visual data flow for UltraWideScaledTail10.

1.3 Layer-by-layer whole-model audit

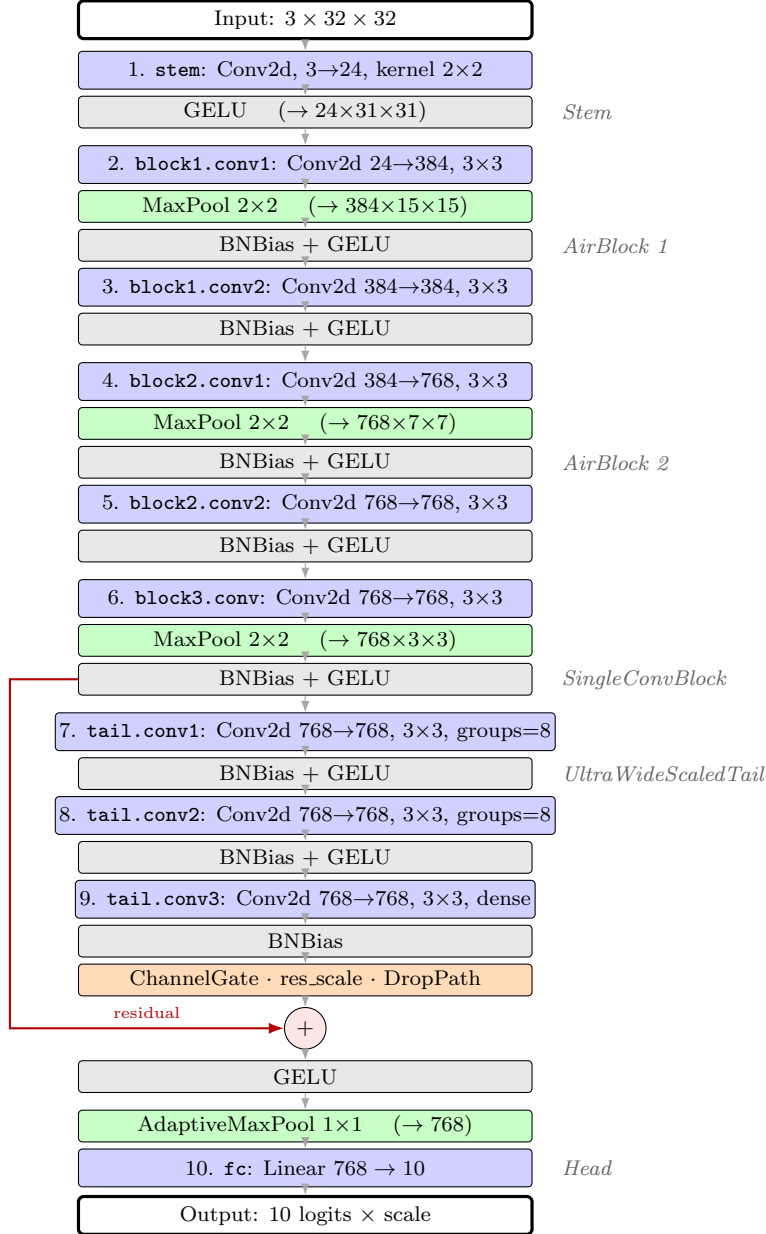
Table 1 gives a forward-pass audit of the whole model, not only the 10 counted weighted layers. The counted project layers are the **Conv2d** and **Linear** rows. The normalization, activation, pooling, channel-gating, residual-scale, and stochastic-depth rows are included so the table represents the actual computation path used by the final network. Reused activation modules are shown at each conceptual use site because this is clearer than hiding repeated GELU applications behind a single module name.

# Layer	Type	Counted?	Shape / setting	Output	Params
1 stem	Conv2d	Yes	24×3×2×2	24×31×31	312
2 stem.act	GELU	No	–	24×31×31	0
3 block1.conv1	Conv2d	Yes	384×24×3×3	384×31×31	82,944
4 block1.pool	MaxPool2d	No	$k=2, s=2$	384×15×15	0
5 block1.bn1	BNBias	No	$C=384$	384×15×15	768
6 block1.act1	GELU	No	–	384×15×15	0
7 block1.conv2	Conv2d	Yes	384×384×3×3	384×15×15	1,327,104
8 block1.bn2	BNBias	No	$C=384$	384×15×15	768
9 block1.act2	GELU	No	–	384×15×15	0
10 block2.conv1	Conv2d	Yes	768×384×3×3	768×15×15	2,654,208
11 block2.pool	MaxPool2d	No	$k=2, s=2$	768×7×7	0
12 block2.bn1	BNBias	No	$C=768$	768×7×7	1,536
13 block2.act1	GELU	No	–	768×7×7	0
14 block2.conv2	Conv2d	Yes	768×768×3×3	768×7×7	5,308,416
15 block2.bn2	BNBias	No	$C=768$	768×7×7	1,536
16 block2.act2	GELU	No	–	768×7×7	0
17 block3.conv	Conv2d	Yes	768×768×3×3	768×7×7	5,308,416
18 block3.pool	MaxPool2d	No	$k=2, s=2$	768×3×3	0
19 block3.bn	BNBias	No	$C=768$	768×3×3	1,536
20 block3.act	GELU	No	–	768×3×3	0
21 tail.conv1	Conv2d	Yes	768×96×3×3, $g=8$	768×3×3	663,552
22 tail.bn1	BNBias	No	$C=768$	768×3×3	1,536
23 tail.act1	GELU	No	–	768×3×3	0
24 tail.conv2	Conv2d	Yes	768×96×3×3, $g=8$	768×3×3	663,552
25 tail.bn2	BNBias	No	$C=768$	768×3×3	1,536
26 tail.act2	GELU	No	–	768×3×3	0
27 tail.conv3	Conv2d	Yes	768×768×3×3	768×3×3	5,308,416
28 tail.bn3	BNBias	No	$C=768$	768×3×3	1,536
29 tail.channel_gate	ChannelGate	No	$C=768$	768×3×3	1,536
30 tail.res_scale	Residual scale	No	1×768×1×1	768×3×3	768
31 tail.drop_path	DropPath	No	$p=0.03$	768×3×3	0
32 tail.act3	GELU	No	residual output	768×3×3	0
33 pool	AdaptiveMax	No	1×1	768×1×1	0
34 fc	Linear	Yes	10×768	10	7,680
Counted weighted-layer parameters (10 rows marked Yes)					21,324,600

Table 1: Whole-model data flow and parameter audit continued.

# Layer	Type	Counted?	Shape / setting	Output	Params
Non-counted normalization/gating/residual parameters (24 rows marked No)					13,056
Total model parameters					21,337,656

The table makes the full path explicit while preserving the project count: there are exactly 10 Conv2d/Linear rows, and all other rows are supporting operations. Most parameters still live in the dense $768 \rightarrow 768$ convolutions, while the normalization, channel-gating, and residual-scale pieces add only 13,056 parameters total.



1.4 Reproducibility run instructions

The final reproducibility script is configured from constants at the top of `train_ultrawidescaledtail10_repro.py`. On an 8-GPU RunPod A40 machine, the current default runs 8 seeds in parallel, one seed per GPU:

```
python -u train_ultrawidescaledtail10_repro.py 2>&1 | tee full_8seed_run.log
```

The default final configuration is:

Setting	Value
Seeds	42 through 49
Epochs per seed	750
Batch size	128
Seed workers	auto, resolving to 8 on an 8-GPU pod
DataLoader workers	32 total, approximately 4 per seed process
Device layout	One independent training process per GPU
Output directory	<code>repro_runs_ultrawidescaledtail10_8seed</code>

The script first ensures that CIFAR-10 is available, then launches one process per GPU. Each seed writes a checkpoint and JSON log. At the end, the script writes `summary.csv` and `summary.json`.

Running on a single-GPU machine. The "auto" setting also handles single-GPU hardware: with one visible GPU, the script runs the 8 seeds sequentially on that GPU, with no code change required. However, each seed took about 93 minutes on an A40, so reproducing all 8 seeds sequentially would take roughly 12 hours. To verify the pipeline in a much shorter time, edit the constants at the top of `train_ultrawidescaledtail10_repro.py` to use a single seed:

```
RUN_SEEDS = (42,)
```

This reduces the run to about 90 minutes on an A40-class GPU while still producing a complete checkpoint, JSON log, and `summary.csv` entry for that seed. The same run command is used in all cases:

```
python -u train_ultrawidescaledtail10_repro.py 2>&1 | tee full_8seed_run.log
```

1.5 Prior selection evidence

Before the final 750-epoch run, I selected the model using prior 500-epoch multi-seed evidence. Among the three strongest depth-limited single-model candidates, `UltraWideScaledTail10` had the highest mean out-of-sample validation accuracy. Ensemble artifacts in the parent experiment folder did score higher, but they combine multiple checkpoints, so I left them out of the single-model result.

Model	Seeds	Mean OOS Val.	Std.	Layers
<code>UltraWideScaledTail10</code>	3	96.97	0.08	10
<code>WideScaledPlus10</code>	3	96.70	0.14	10
<code>DeepScaled10</code>	3	96.37	0.14	10

My retained seed-level evidence for the selected model was:

Seed	Selected OOS Val.	Best Raw	Best EMA	Selected Epoch
42	96.94	96.88	96.94	459
43	97.06	97.06	97.02	481
44	96.90	96.90	96.90	486

The selected checkpoints occurred late in training, around epochs 459–486. This supports the final decision to increase the reproducibility schedule to 750 epochs.

1.6 Final 8-seed results

The completed 8-seed RunPod sweep is summarized below from `summary.csv`. The log file `full_8seed_run.log` confirms that all eight seed jobs finished and wrote the sweep summary. The `test_acc` column in the CSV is blank because this run used validation selection only (`RUN_EVAL_TEST_AT_END = False`).

Seed	Ckpt.	Selected Val.	Epoch	Best Raw	Best EMA	Wall Min.
42	raw	96.90	734	96.90@734	96.88@747	93.7
43	ema	96.72	744	96.70@731	96.72@744	93.1
44	raw	96.74	715	96.74@715	96.70@734	93.5
45	raw	96.92	742	96.92@742	96.92@742	92.2
46	raw	96.74	686	96.74@686	96.68@736	92.0
47	ema	96.88	750	96.82@735	96.88@750	92.9
48	ema	96.84	745	96.78@725	96.84@745	91.3
49	raw	96.98	716	96.98@716	96.90@740	93.8

Across the eight seeds, the mean selected validation accuracy was 96.84% with sample standard deviation 0.10. The minimum selected validation accuracy was 96.72%, the maximum was 96.98%, and the average per-seed wall time was 92.8 minutes. Broken out by checkpoint type at each seed’s best epoch, the average best-raw accuracy across the eight seeds was 96.82% and the average best-EMA accuracy across the eight seeds was also 96.82%. The per-seed mix, taking whichever of raw or EMA was higher for each seed, averages to 96.84%, matching the selected-validation mean.

2 How and Why the Model Was Improved

I organized this section around the actual custom blocks in the final model. Each block is shown in abbreviated code form, followed by the design reason and the alternative that was rejected.

2.1 Full architecture: wide trunk plus late residual tail

Listing 1: Submitted model skeleton

```
class UltraWideScaledTail10(nn.Module):
    def __init__(self, num_classes: int = 10, scale: float = 1 / 9):
        super().__init__()
        self.stem = nn.Conv2d(3, 24, kernel_size=2, padding=0, bias=True)
        self.stem_act = nn.GELU()
        self.block1 = AirBlock(24, 384, extra=False, pool=True)
        self.block2 = AirBlock(384, 768, extra=False, pool=True)
        self.block3 = SingleConvBlock(768, 768, pool=True)
        self.tail = UltraWideScaledTail(768)
        self.pool = nn.AdaptiveMaxPool2d(1)
        self.fc = nn.Linear(768, num_classes, bias=False)
        self.scale = scale

    def forward(self, x):
        x = self.stem_act(self.stem(x))
        x = self.block1(x)
        x = self.block2(x)
        x = self.block3(x)
        x = self.tail(x)
        x = self.pool(x).flatten(1)
        return self.fc(x) * self.scale
```

I made the model wider instead of deeper because the project has a strict 10-weighted-layer budget. Wide Residual Networks showed that increasing width can be a better use of capacity than simply adding more depth on CIFAR-style tasks [2]. The competing `DeepScaled10` model had fewer parameters and more depth-like structure, but its 3-seed mean validation accuracy was lower (96.37 versus 96.97). The selected architecture therefore spends the limited layer budget on a high-capacity trunk and only adds extra refinement after the image has been reduced to a small spatial map.

2.2 AirBlock and SingleConvBlock: simple convolutional trunk

Listing 2: Main trunk blocks

```
class AirBlock(nn.Module):
    def __init__(self, cin, cout, pool=True):
        super().__init__()
        self.conv1 = nn.Conv2d(cin, cout, 3, padding=1, bias=False)
        self.pool = nn.MaxPool2d(2) if pool else nn.Identity()
        self.bn1 = BNBias(cout)
        self.conv2 = nn.Conv2d(cout, cout, 3, padding=1, bias=False)
        self.bn2 = BNBias(cout)
        self.act = nn.GELU()

    def forward(self, x):
        x = self.act(self.bn1(self.pool(self.conv1(x))))
        return self.act(self.bn2(self.conv2(x)))

class SingleConvBlock(nn.Module):
    def __init__(self, cin, cout, pool=True):
        super().__init__()
        self.conv = nn.Conv2d(cin, cout, 3, padding=1, bias=False)
        self.pool = nn.MaxPool2d(2) if pool else nn.Identity()
        self.bn = BNBias(cout)
        self.act = nn.GELU()

    def forward(self, x):
        return self.act(self.bn(self.pool(self.conv(x))))
```

I used these blocks over transformer-style or MLP-style blocks because CIFAR-10 images are small and local convolutional features are strong early in training. Pooling is separate and parameter-free, so downsampling does not consume another weighted layer. The activation is GELU rather than ReLU: ReLU is cheaper, but GELU gives a smooth nonlinearity without adding parameters or counted layers [6]. This was useful because the model has very few nonlinear stages under the layer limit.

2.3 BNBias: normalization without extra trainable scale

Listing 3: Frozen-scale batch normalization

```
class BNBias(nn.BatchNorm2d):
    def __init__(self, c: int, momentum: float = 0.4):
        super().__init__(c, momentum=momentum, affine=True)
        nn.init.ones_(self.weight)
        self.weight.requires_grad = False
```

BNBias keeps the stabilizing effect of batch normalization while freezing the multiplicative scale parameter. Batch normalization is used because it improves optimization by normalizing intermediate feature distributions [5]. I chose this over no normalization because the very wide layers were harder to train stably without normalization. I also chose it over adding heavier normalization or attention layers because the main capacity should stay in the counted convolutions.

2.4 ChannelGate: lightweight channel attention

Listing 4: ChannelGate

```
class ChannelGate(nn.Module):
    def __init__(self, channels: int, init_scale: float = 0.0):
        super().__init__()
        self.weight = nn.Parameter(torch.full((1, channels, 1, 1), init_scale))
        self.bias = nn.Parameter(torch.zeros(1, channels, 1, 1))

    def forward(self, x):
        avg = x.mean(dim=(2, 3), keepdim=True)
        scale = torch.sigmoid(avg * self.weight + self.bias)
        return x * (1.0 + scale)
```

I used this block instead of a full Squeeze-and-Excitation MLP. SE networks showed that channel-wise recalibration can improve convolutional features [4], but a standard SE block would add extra `Linear` or `Conv2d` layers. Under my layer-count rule, those layers would count against the model budget. `ChannelGate` keeps the useful idea, global channel statistics controlling channel strength, while avoiding additional counted weighted layers.

2.5 DropPath: residual-path regularization

Listing 5: DropPath

```
class DropPath(nn.Module):
    def __init__(self, drop_prob: float = 0.0):
        super().__init__()
        self.drop_prob = float(drop_prob)

    def forward(self, x):
        if self.drop_prob == 0.0 or not self.training:
            return x
        keep_prob = 1.0 - self.drop_prob
        shape = (x.shape[0],) + (1,) * (x.ndim - 1)
        mask = x.new_empty(shape).bernoulli_(keep_prob)
        return x * mask.div(keep_prob)
```

`DropPath` was my choice over ordinary dropout because the main overfitting risk is the large residual tail, not a dense classifier head. Dropout reduces co-adaptation [8, 9]; stochastic depth applies the same idea to residual branches [10]. In this model, the residual tail is randomly weakened during training while the full path is used at inference.

2.6 UltraWideScaledTail: grouped residual refinement

Listing 6: Low-resolution residual tail

```
class UltraWideScaledTail(nn.Module):
    def __init__(self, channels: int, groups: int = 8):
        super().__init__()
        self.conv1 = nn.Conv2d(channels, channels, 3, padding=1,
                                groups=groups, bias=False)
        self.bn1 = BNBias(channels)
        self.conv2 = nn.Conv2d(channels, channels, 3, padding=1,
                                groups=groups, bias=False)
        self.bn2 = BNBias(channels)
        self.conv3 = nn.Conv2d(channels, channels, 3, padding=1, bias=False)
        self.bn3 = BNBias(channels)
        self.res_scale = nn.Parameter(torch.zeros(1, channels, 1, 1))
        self.channel_gate = ChannelGate(channels)
        self.drop_path = DropPath(0.03)
        self.act = nn.GELU()

    def forward(self, x):
        residual = x
        x = self.act(self.bn1(self.conv1(x)))
        x = self.act(self.bn2(self.conv2(x)))
        x = self.bn3(self.conv3(x))
        x = self.channel_gate(x)
        return self.act(residual + self.drop_path(self.res_scale * x))
```

I placed the tail after the resolution has fallen to $768 \times 3 \times 3$. I chose this over adding early high-resolution convolutions because late convolutions are much cheaper in activation memory. The residual addition follows the ResNet idea of learning a correction to an existing representation [1]. The first two tail convolutions are grouped to reduce parameter and compute cost while retaining multiple parallel transformations, following the ResNeXt cardinality idea [3]. The final dense convolution mixes information across all channels. The residual scale is initialized at zero so the tail starts as a conservative correction instead of disrupting the trunk at the start of training.

2.7 Initialization

Listing 7: Initialization pattern

```
def init_dirac_(conv: nn.Conv2d) -> None:
    nn.init.dirac_(conv.weight.data, groups=conv.groups)

def _apply_init(self) -> None:
    for block in (self.block1, self.block2):
        init_dirac_(block.conv1)
        init_dirac_(block.conv2)
    init_dirac_(self.block3.conv)
    init_dirac_(self.tail.conv1)
    init_dirac_(self.tail.conv2)
    init_dirac_(self.tail.conv3)
    nn.init.kaiming_normal_(self.fc.weight, nonlinearity="linear")
```

I used Dirac initialization for convolutional layers because it starts compatible layers close to an identity-like mapping. This matches the residual design: begin from a stable path and learn useful deviations. The classifier uses Kaiming initialization, which was designed for deep rectifier networks and remains a standard initialization choice for convolutional classifiers [7].

2.8 Training recipe choices

Technique	Used in script	Reason
RandAugment and random erasing	RandAugment(2,14) plus RandomErasing(p=0.25)	Stronger image variation without a separate policy search; random erasing forces less reliance on one local cue [11, 12].
Mixup	RUN_MIXUP = 0.2	Smooths labels between examples and reduces overconfident memorization [13].
Label smoothing	RUN_LABEL_SMOOTHING = 0.1	Prevents the classifier from training against fully hard one-hot targets [14].
SGD with warmup and cosine decay	SGD(..., nesterov=True) with warmup cosine schedule	Warmup avoids unstable early updates; cosine decay gives smaller late updates for convergence [15, 16].
EMA checkpointing	AveragedModel(..., ema_decay=0.999)	Weight averaging often improves validation stability and can select a flatter solution [17, 18].
Mixed precision and channels-last	CUDA bfloat16 autocast and channels-last tensors	Improves GPU throughput and memory behavior without changing the architecture [19, 20].

3 Attempts I Did Not Keep

I tried or considered the following ideas during model development but did not keep them in my final single-model submission. I included more than ten because they show the iteration path, not just the final result. I treated these attempts as ablations: each one tested whether a different way of spending the 10-layer budget would give a better accuracy, simplicity, or reproducibility tradeoff.

#	Attempt	What I was testing	Why I did not keep it
1	DeepScaled10	A more depth-oriented 10-layer candidate with fewer parameters than the final model.	Its prior 3-seed mean validation accuracy was 96.37, below 96.97 for UltraWideScaledTail10 . This told me that extra depth-like structure was not as valuable as width under the same layer limit.
2	WideScaledPlus10	A close 10-layer width-based alternative with a different scaling and tail balance.	It was competitive, but its prior 3-seed mean was 96.70 with higher spread than the selected model. I kept the stronger and more consistent UltraWideScaledTail10 result.
3	Plain deeper scaling	Using the limited layer budget to add more sequential transformations instead of increasing channel capacity.	The tests did not beat the width-first direction. With only 10 counted layers, every extra convolution reduced the budget available for wide feature maps.
4	Smaller parameter model preference	Choosing the smaller model simply because it was lighter and easier to run.	DeepScaled10 was much smaller, but it gave lower validation accuracy. I decided that parameter count alone was not the right objective as long as the final model still fit the hardware and layer constraint.
5	Historical UltraWide7	An earlier 7-layer wide model that tested whether fewer counted layers could still get close to the final accuracy.	It reached 95.14 official test accuracy, which was useful progress but clearly below the later 10-layer evidence. I used it as motivation to keep the wide trunk but spend the remaining layers more carefully.
6	Historical WideScaled10	An earlier width-plus-tail 10-layer direction.	It reached 94.83 official test accuracy, below later variants. The result suggested that the general idea was promising, but the exact scaling and late tail needed improvement.
7	GhostScaled10	A compact ghost-style module intended to get cheaper feature expansion.	It reached 94.52 official test accuracy and did not justify replacing the direct wide convolutions. The cheaper module saved capacity, but the wide trunk learned stronger CIFAR-10 features.
8	CoordAtt10	A coordinate-attention style model that tested whether explicit spatial/channel attention would help enough to replace width.	It also reached 94.52 official test accuracy. I did not keep it because the added attention idea did not outperform the simpler wide convolutional trunk.

#	Attempt	What I was testing	Why I did not keep it
9	Learned feature-concatenation ensemble	Combining features from several trained models before classification.	Did not respond to training as I hoped, I realized an ensemble of separately trained model would be better, but that would entail adding an 11th ensemble layer to the ensemble of 10-Layer models.
10	Weighted logit ensemble	A simpler ensemble that combined model predictions rather than internal features.	It improved complementarity, but it did not fit my single-model goal and exceeded the strict total-layer interpretation. I kept the insight that the models made different errors, but not the ensemble itself.
11	Keeping all top-three models in the notebook	Leaving <code>UltraWideScaledTail10</code> , <code>WideScaledPlus10</code> , and <code>DeepScaled10</code> all in the final notebook.	This slowed iteration and made the final comparison less focused. I streamlined the notebook around the one model I actually selected, while reporting the others as selection evidence.
12	Short training as final evidence	Using short runs to choose the final model and avoid long GPU sweeps.	Short runs were useful smoke tests, but the selected checkpoints appeared around epochs 459–486 in prior evidence and around epochs 686–750 in the final 8-seed sweep. I therefore used long training for the result I report.

These attempts were useful because they narrowed the final design principle: within a strict 10-layer budget, the best single model was not the deepest or smallest model, but the model that combined a wide feature trunk with a low-resolution residual tail.

4 Conclusion

`UltraWideScaledTail10` is my final selected single model. It satisfies the 10-weighted-layer budget, has 21,337,656 parameters, and I selected it because it had the best prior multi-seed out-of-sample validation performance among the tested single-model candidates. I ran the final RunPod configuration for 8 seeds and 750 epochs on 8 A40 GPUs, with one independent seed process per GPU.

References

- [1] K. He, X. Zhang, S. Ren, and J. Sun. Deep Residual Learning for Image Recognition. *CVPR*, 2016. <https://arxiv.org/abs/1512.03385>.
- [2] S. Zagoruyko and N. Komodakis. Wide Residual Networks. *BMVC*, 2016. <https://arxiv.org/abs/1605.07146>.
- [3] S. Xie, R. Girshick, P. Dollar, Z. Tu, and K. He. Aggregated Residual Transformations for Deep Neural Networks. *CVPR*, 2017. <https://arxiv.org/abs/1611.05431>.
- [4] J. Hu, L. Shen, S. Albanie, G. Sun, and E. Wu. Squeeze-and-Excitation Networks. *CVPR*, 2018. <https://arxiv.org/abs/1709.01507>.
- [5] S. Ioffe and C. Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. *ICML*, 2015. <https://arxiv.org/abs/1502.03167>.
- [6] D. Hendrycks and K. Gimpel. Gaussian Error Linear Units (GELUs). arXiv preprint, 2016. <https://arxiv.org/abs/1606.08415>.
- [7] K. He, X. Zhang, S. Ren, and J. Sun. Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification. *ICCV*, 2015. <https://arxiv.org/abs/1502.01852>.
- [8] G. E. Hinton, N. Srivastava, A. Krizhevsky, I. Sutskever, and R. R. Salakhutdinov. Improving Neural Networks by Preventing Co-adaptation of Feature Detectors. arXiv preprint, 2012. <https://arxiv.org/abs/1207.0580>.
- [9] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014. <https://www.jmlr.org/papers/v15/srivastava14a.html>.
- [10] G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Q. Weinberger. Deep Networks with Stochastic Depth. *ECCV*, 2016. <https://arxiv.org/abs/1603.09382>.
- [11] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le. RandAugment: Practical Automated Data Augmentation with a Reduced Search Space. *CVPR Workshops*, 2020. <https://arxiv.org/abs/1909.13719>.
- [12] Z. Zhong, L. Zheng, G. Kang, S. Li, and Y. Yang. Random Erasing Data Augmentation. *AAAI*, 2020. <https://arxiv.org/abs/1708.04896>.
- [13] H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz. mixup: Beyond Empirical Risk Minimization. *ICLR*, 2018. <https://arxiv.org/abs/1710.09412>.
- [14] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna. Rethinking the Inception Architecture for Computer Vision. *CVPR*, 2016. <https://arxiv.org/abs/1512.00567>.
- [15] P. Goyal, P. Dollar, R. Girshick, P. Noordhuis, L. Wesolowski, A. Kyrola, A. Tulloch, Y. Jia, and K. He. Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour. arXiv preprint, 2017. <https://arxiv.org/abs/1706.02677>.
- [16] I. Loshchilov and F. Hutter. SGDR: Stochastic Gradient Descent with Warm Restarts. *ICLR*, 2017. <https://arxiv.org/abs/1608.03983>.

- [17] P. Izmailov, D. Podoprikin, T. Garipov, D. Vetrov, and A. G. Wilson. Averaging Weights Leads to Wider Optima and Better Generalization. *UAI*, 2018. <https://arxiv.org/abs/1803.05407>.
- [18] A. Tarvainen and H. Valpola. Mean Teachers are Better Role Models: Weight-Averaged Consistency Targets Improve Semi-Supervised Deep Learning Results. *NeurIPS*, 2017. <https://arxiv.org/abs/1703.01780>.
- [19] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu. Mixed Precision Training. *ICLR*, 2018. <https://openreview.net/pdf?id=r1gs9JgRZ>.
- [20] PyTorch Contributors. Channels Last Memory Format in PyTorch. PyTorch documentation. https://docs.pytorch.org/tutorials/intermediate/memory_format_tutorial.html.